

Risposta di claude al seguente messaggio:

pre processing con del colore (bianco e nero)? rimarcare i bordi? altre opzioni di pre processing per diminuire il carico di lavoro sul processore ai;

esperienza parallela interfaccia utente, da rimandare;

telecamera ipoteticamente su un altezza di 2/3 metri, come su un semaforo; capire quindi poi come leggere la targa nel migliore dei modi.

le 6 categorie vanno bene; magari rilevare mezzi lavorativi come ambulanza, pompieri;

velocità veicoli massima 50/60 km/h;

evitare chiamate api a gemini per la lettura della targa, farlo però come prova per confronto con ocr, andando a vedere su quanti errori di ocr gemini invece le fa bene. Quindi non ci sono vincoli di connettività

Provare a capire se si può usare database delle motorizzazioni per le targhe e capire proprio che veicolo è, proprio il modello.

output per ora va bene il json quindi con un conteggio dei veicoli, oltre che agli id dei veicoli con categoria e targa; per ora va bene output sul dispositivo.

La call è finita, ci ha detto che abbiamo fatto un bel lavoro e appunto ora va continuato ad essere raffinato; ci sentiremo tra 2 settimane

categorie vanno bene, magari con i prossimi test con targhe italiane (per ora va bene concentrarci solo su queste) fare proprio delle metriche di accuratezza;

Riepilogo requisiti emersi

Scenario operativo

- Camera a 2-3m di altezza (tipo semaforo) → angolazione dall'alto, targa visibile in determinati momenti
- Velocità veicoli max 50-60 km/h
- Nessun vincolo di connettività
- Output JSON on-device con conteggio veicoli + ID + categoria + targa

Categorie veicoli

- Le 6 attuali vanno bene
- Aggiungere mezzi di emergenza (ambulanza, pompieri, forse polizia/carabinieri)
- Test su targhe italiane con metriche di accuratezza

Targhe

- Solo italiane per ora
- No Gemini API in produzione, ma sì come benchmark di confronto con EasyOCR

Da esplorare

- Pre-processing immagini (B/N, edge enhancement, ecc.) per alleggerire il carico AI
 - Database motorizzazione per arricchire l'output con modello del veicolo
 - UI parallela (rimandata)
-

Punti da approfondire / decisioni tecniche

1. Pre-processing – attenzione al trade-off

Il pre-processing in scala di grigi o edge enhancement ha senso per l'OCR (EasyOCR beneficia di immagini binarizzate, contrasto alto), ma **non per Hailo-8**: i modelli YOLO sono compilati con input RGB fisso a una certa risoluzione. Cambiare l'input richiede ri-training e ri-compilazione.

Quindi io distinguerei:

- **Pre-processing pre-OCR** (sì, ha senso): grayscale, thresholding adattivo, denoising sul crop della targa prima di passarla a EasyOCR. Questo riduce errori, non carico.
- **Pre-processing pre-Hailo** (rischioso): non si può cambiare facilmente.
- **Per ridurre carico CPU**: meglio agire sul *frame skipping* (processare 1 frame su N invece di tutti) o sulla risoluzione di input. Domanda da farsi: a 50 km/h, quanti frame al secondo servono davvero?

2. Camera a 2-3m dall'alto – angolazione critica

Targa visibile bene solo quando il veicolo è abbastanza vicino e frontalmente. Lateralmente o quando è sotto la camera, la targa scompare. Questo significa:

- La targa va letta in una *finestra temporale specifica* del passaggio del veicolo
- Il majority voting su tracker ID che già abbiamo è esattamente la strategia giusta
- Possiamo aggiungere uno score di confidenza basato sull'area della bbox targa (più grande = più affidabile)

3. Database motorizzazione – fattibilità

Il database del Pubblico Registro Automobilistico (PRA/ACI) è accessibile ma:

- L'accesso pubblico è limitato e richiede convenzioni specifiche
- I servizi commerciali (es. Visura.it) costano per query

- Senza convenzione, sarebbe da capire se l'azienda ha già un accordo o vuole stipularne uno

Vale la pena chiederglielo. Per ora possiamo predisporre un *campo "model" nell'output JSON* lasciato vuoto/null, in modo che il sistema sia pronto ad accogliere l'arricchimento quando arriva.

4. Gemini come benchmark – ottimo approccio

Idea concreta: su un sottoinsieme di plate crops, lanciare in parallelo EasyOCR e Gemini API, salvare entrambi gli output, e calcolare:

- Accuratezza carattere per carattere
- Errori per tipologia (Z/7, G/6, ecc.)
- Tempo di risposta

Questo ci dà un report di confronto pulito.

Roadmap proposta per le 2 settimane

Settimana 1

1. Estendere il dataset/categorie con mezzi di emergenza (cercare immagini su OpenImages, Roboflow)
2. Aggiungere pre-processing pre-OCR sul crop targa (grayscale + adaptive threshold + denoise)
3. Implementare frame skipping configurabile
4. Iniziare raccolta dati reali su targhe italiane per metriche

Settimana 2

5. Test nano vs tiny_yolov4 per plate detection (già pianificato)
 6. Script benchmark EasyOCR vs Gemini su sample di plate crops
 7. Calcolare metriche di accuratezza:
 - Precision/Recall per classificazione veicoli
 - Character Error Rate (CER) per OCR targhe
 - % di targhe lette correttamente per veicolo passante
 8. Predisporre output JSON con campo "model" placeholder per database motorizzazione
-

Vuoi che partiamo da qualcosa di specifico tra questi? Per esempio potrei già scriverti lo script di benchmark EasyOCR vs Gemini, o aiutarti a integrare il pre-processing pre-OCR nel codice esistente.

Risposta MIA a Claude

parliamo dei punti da approfondire

1. si è quello che faremo, tramite google cloud che permette di avere un account con 300€ a disposizione per chiamate api a gemini
2. l'azienda non ha accordi; dovremmo vedere se esistono dei database gratuiti della motorizzazione italiana, in cui appunto, dopo aver letto e scelto (majority voting) la targa, la si va a cercare in quel db per vedere se appunto la categoria del veicolo corrisponde e si vede inoltre il modello nello specifico; Sicuramente utile per rilevare i mezzi lavorativi, anche per il fatto delle targhe specifiche.
3. si va capito come fare questa cosa, se serve stretchare l'immagine in qualche modo per permettere una miglior lettura della targa o un rilevamento più efficace dei veicoli e categoria
4. so che per quanto riguarda il file .hes della small andrebbe riaddestrato interamente il modello per farlo funzionare con le immagini b/n; però forse con un edge enhancement no, per capire la categoria del veicolo o anche per il bbox della targa; pre-ocr ha sicuramente senso fare dei test per capire cosa funziona nel migliore dei modi

Detto ciò ho un'altra cosa da dirti, il mio collega federico ha addestrato due yolov11nano per la plate detection su dataset uc3m e v1n; andranno testati insieme alla nostra tinyyolov4 per vedere quale dà risultati migliori per la plate detection. Useremo un dataset estraneo per valutare i 3 modelli hes e anche confrontandoli con i modelli pt delle yolov11n su ec3m e v1n

Seconda risposta sua

Ottimo, ora abbiamo un piano molto concreto. Ti rispondo punto per punto e poi parliamo del benchmark a 3 modelli.

Sui punti di approfondimento

1. Gemini benchmark via Google Cloud

I 300€ di credito gratuito sono perfetti per questo tipo di benchmark, ne avanza parecchio. Ti consiglio di usare **Gemini 2.5 Flash** o **Gemini 2.0 Flash** per il rapporto qualità/prezzo — sono ottimi sull'OCR e costano una frazione del Pro. Quando vuoi, ti preparo lo script che:

- Prende i plate crops dal nostro output
- Lancia EasyOCR e Gemini in parallelo
- Salva i risultati in un CSV con ground truth (quando disponibile)
- Calcola Character Error Rate, accuratezza per carattere e tempi di risposta

2. Database motorizzazione italiana – realtà scomoda

Bisogna essere onesti su questo punto: **non esiste un database pubblico gratuito completo del PRA/Motorizzazione**. L'ACI ha un servizio online di visure ma è a pagamento (1-3€ a visura) e richiede registrazione. Esistono però alcune alternative interessanti:

- **Servizio "Verifica targa" del Portale dell'Automobilista (MIT)** – gratuito, restituisce info su revisione/assicurazione data una targa. Limitato ma usabile.
- **API commerciali** (es. CarPlate, OpenANPR, vari aggregatori) – a pagamento, costano qualche centesimo per query.
- **Datasets open** tipo Kaggle/HuggingFace di marca-modello → ma sono per *riconoscimento visivo* da immagine, non per lookup da targa.

La mia proposta concreta: in parallelo al benchmark OCR, posso preparare un piccolo proof-of-concept che interroga il Portale dell'Automobilista a partire da una targa e ne estrae le info disponibili. Così verifichiamo se è una via percorribile gratuita prima di proporre soluzioni a pagamento all'azienda.

3. Lettura targa con camera dall'alto – strategia

Lo stretching dell'immagine è un'opzione, ma di solito è meglio:

- **Perspective correction** sul crop targa una volta detettato (4 punti → rettangolo) per "raddrizzarla" se inclinata
- **Super-resolution sul crop** prima dell'OCR – esistono modelli leggeri tipo ESRGAN o anche metodi più semplici (bicubic + sharpen) che migliorano molto la leggibilità di targhe lontane/piccole
- Sul *vehicle detection* invece, da 2-3m la categoria si distingue bene anche senza stretching, perché i veicoli sono visti in prospettiva quasi normale

4. Edge enhancement / pre-processing pre-Hailo – ottimo intuito

Hai ragione: edge enhancement è meno invasivo del B/N puro perché preserva i 3 canali RGB. In teoria si può applicare prima di Hailo *senza ricompilare*, perché l'input shape resta uguale. Però attenzione: il modello è stato addestrato su immagini "naturali", quindi se gli passi qualcosa di troppo elaborato potrebbe degradare le performance invece di migliorarle.

Soluzione pulita: testare empiricamente. Possiamo creare uno script che processa lo stesso video con N varianti di pre-processing (originale, edge enhancement leggero, edge enhancement forte, contrast boost, ecc.) e confronta mAP e numero di detection per categoria.

Sul benchmark 3 modelli plate detection

Ottimo che Federico abbia addestrato due nano su UC3M e V1N! Questo è il setup di confronto:

Modelli da testare (in formato HEF su Hailo-8)

1. `tiny_yolov4_license_plates.hef` (Hailo Model Zoo, già funzionante)

2. `yolov11n_uc3m.hef` (Federico, dataset UC3M)
3. `yolov11n_v1n.hef` (Federico, dataset V1N)

Modelli di riferimento (PT)

- `yolov11n_uc3m.pt`
- `yolov11n_v1n.pt`

Servono per capire **quanta accuratezza si perde nella compilazione a HEF** (la quantizzazione INT8 introduce sempre un calo, a volte significativo).

Dataset di valutazione

Deve essere *estraneo* a entrambi i training (UC3M e V1N), altrimenti il confronto è inquinato.

Suggerimenti:

- Italian License Plate Dataset su Kaggle/Roboflow
- Un sottoinsieme di immagini reali dal vostro video del parcheggio, annotate a mano (bastano 50-100 plates per metriche decenti)
- CCPD (cinese, ma ottimo per stress-test di generalizzazione)

L'opzione 2 è la più rilevante per il caso d'uso reale di NGS.

Metriche da calcolare

- **mAP50** e **mAP50-95** per ogni modello
- **Precision/Recall** a IoU 0.5
- **Tempo di inferenza** per frame (i nano su Hailo dovrebbero essere più veloci di `tiny_yolov4`)
- **Calo da PT a HEF** per i due yolov11n (delta mAP50)
- **Tasso di false positive** – importante perché in produzione una targa "inventata" è peggio di una targa "mancata"

Posso prepararti uno script di benchmark che genera tutto questo in un report unico. Vuoi che parta da questo o preferisci che inizi prima dal benchmark OCR EasyOCR vs Gemini?